



DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

With Intuition • Approach • Dry Run • Optimal Solution • Interview Tips



ROADMAP

- ✓ Arrays & Hashing
- ✓ Two Pointers / Sliding Window
- ✓ Binary Search
- ✓ Linked List
- ✓ Trees
- ✓ Graphs
- ✓ Dynamic Programming
- ✓ Backtracking
- ✓ Heap / Stack / Queue
- ✓ Advanced Problems

Master Patterns, not just individual problems.



HOW FAANG INTERVIEWS WORK

1. Online Assessment
(DSA + Puzzles)2. Technical Rounds
(2-4 Coding Rounds)3. Onsite / Virtual Rounds
(System Design + DSA)4. Behavioral Round
(Leadership, Ownership)

5. Offer



HOW TO APPROACH ANY DSA QUESTION

- 1 Understand the problem clearly (I/O, constraints)
- 2 Think of Brute Force
- 3 Look for patterns (Can I optimize this?)
- 4 Choose the right data structure / algorithm
- 5 Dry run with small examples
- 6 Code + analyze complexity

IMPORTANT THINGS TO REMEMBER

- ✓ Understand **WHY** a solution works.
- ✓ Always analyze **Time & Space Complexity**.
- ✓ Write clean, optimized & bug free code.
- ✓ Communicate your thought process.

PROBLEMS 1 - 5 (ARRAYS)

1 Two Sum

Problem

Given an array nums and a target, return indices of the two numbers such that they add up to target.

Intuition

For each number, find its complement (target - num) in the array.

Optimal Approach

Use HashMap to store number → index.

Dry Run

nums = [2, 7, 11, 15], target = 9

i	0	1	2	3
nums[i]	2	7	11	15
map	{}	{2:0}	{2:0, 7:1}	{2:0, 7:1}

At i = 1, complement = 2 found at index 0.

Answer = [0, 1]

Complexity

Time: O(n)

Space: O(n)

FAANG Tip

Always try HashMap for complement based problems.

2 Best Time to Buy & Sell Stock

Problem

Given prices[i], return max profit by buying and selling the stock once.

Intuition

Track the minimum price so far and maximize profit at each step.

Optimal Approach

One pass: keep minPrice and maxProfit.

Dry Run

prices = [7, 1, 5, 3, 6, 4]

price	7	5	3	6	4
minPrice	7	1	1	1	1
maxProfit	0	0	4	5	5

Answer = 5

Complexity

Time: O(n)

Space: O(1)

FAANG Tip

Think in terms of what info you must keep while traversing.

3 Contains Duplicate

Problem

Given an array, return true if any element appears at least twice.

Intuition

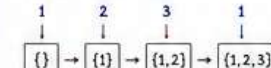
If we see a number again, it's a duplicate.

Optimal Approach

Use HashSet to track seen numbers.

Dry Run

nums = [1, 2, 3, 1]



Duplicate Found

Complexity

Time: O(n)

Space: O(n)

FAANG Tip

HashSet / Set is your best friend for duplicate problems.

4 Product Except Self

Problem

Return an array answer where answer[i] is the product of all elements except nums[i]. Do it without division.

Intuition

For each index, product of left side × right side.

Optimal Approach

Use prefix & suffix products. Two passes.

Dry Run

nums = [1, 2, 3, 4]

prefix :: 1, 1, 2, 6 (left products)

suffix :: [24, 12, 4, 1] (right products)

answer :: [24, 12, 8, 6]

Complexity

Time: O(n)

Space: O(1)
(ignoring output)

FAANG Tip

Look for patterns before jumping to advanced DS.

5 Maximum Subarray (Kadane's Algorithm)

Problem

Find the contiguous subarray with the largest sum.

Intuition

At any point, either extend the existing subarray or start a new subarray from current element.

Optimal Approach

Use Kadane's Algorithm.

Dry Run

nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

i	0	1	2	3	4	5	6	7	8
nums[i]	-2	1	-3	-1	2	1	-5	4	
currentSum	-2	1	-2	4	5	6	1	5	
maxSum	-2	1	1	4	4	6	6	6	6

Answer = 6

Complexity

Time: O(n)

Space: O(1)

FAANG Tip

Kadane's is asked in many forms. Master the intuition!

COMMON PATTERNS (REMEMBER!)

- Sliding Window
- Two Pointers
- Fast & Slow Pointer
- DFS / BFS
- Binary Search
- DP (Memo + Tabulation)
- Backtracking



TIME COMPLEXITY CHEAT SHEET

- O(1) → Constant
- O(log n) → Logarithmic
- O(n) → Linear
- O(n log n) → Linearithmic
- O(n²) → Quadratic
- O(2ⁿ) → Exponential



INTERVIEW CHECKLIST ✓

- ✓ Clarify the problem
- ✓ Handle edge cases
- ✓ Think of brute force
- ✓ Optimize
- ✓ Dry run
- ✓ Code
- ✓ Analyze complexity

FINAL THOUGHT 💡

"A great developer is not one who remembers all solutions, but one who can think of the right approach for any problem."

— Keep Practicing!





PROBLEMS 6 - 10 (ARRAYS & SLIDING WINDOW)

6 Longest Substring Without Repeating Characters

Problem

Given a string s , find the length of the longest substring without repeating chars.

Intuition

Use a sliding window + HashMap to keep track of last seen index.

Optimal Approach

Expand the window, if duplicate found, move left pointer to the right of last occurrence.

Dry Run $s = \text{"abcabcbb"}$

i	0	1	2	4	4	6	7
s[i]	a	b	c	a	b	b	b
l	0	0	0	1	2	3	6
maxLen	1	2	3	3	3	3	3

a	b	c	a	b	c	b	b
0	1	2	3	4	5	6	7

Diagram showing the sliding window [l, r] and the current character being added. The window expands until a duplicate is found, then contracts from the left.

Answer = 3

7 Maximum Average Subarray I (Size K)

Problem

Given an array $nums$ and an integer k , find the maximum average of any subarray of size k .

Intuition

Use fixed size sliding window and track the sum.

Optimal Approach

Calculate sum of first window, then slide and update sum.

Dry Run $nums = [1, 12, -5, -6, 50, 3], k = 4$

Window 1: $[1, 12, -5, -6]$ Sum = 2 Avg = 0.50

Window 2: $[12, -5, -6, 50]$ Sum = 51 Avg = 12.75

Window 3: $[-5, -6, 50, 3]$ Sum = 42 Avg = 10.50

Complexity

Time: $O(n)$

Space: $O(1)$

FAANG Tip

Fixed window size problems are very common in interviews!

Answer = 12.75

8 Longest Consecutive Sequence

Problem

Given an unsorted array of integers, return the length of the longest consecutive elements sequence.

Intuition

Use HashSet for $O(1)$ lookups. Only start counting from numbers that don't have a previous consecutive number.

Optimal Approach

Add all numbers to set. For each number, if $(num-1)$ not in set $\rightarrow$ start counting.

Dry Run $nums = [100, 4, 200, 1, 3, 2]$

Set = $\{100, 4, 200, 1, 3, 2\}$

Start from 1 (since 0 not in set)

1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4

Length = 4

Answer = 4

9 Container With Most Water

Problem

Given n non-negative integers $a_1, a_2, \dots, a_n$, where each represents a point at (i, a_i) . Find two lines that together with x-axis forms a container that holds the most water.

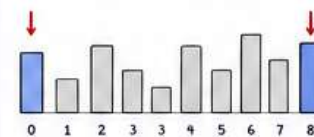
Intuition

Brute force is $O(n^2)$. Use two pointer approach to optimize.

Optimal Approach

Use two pointers from both ends. Always move the pointer with smaller height.

Dry Run $height = [1, 8, 6, 2, 5, 4, 8, 3, 7]$



$l=0, r=8 \rightarrow \text{area} = \min(1, 7) * 8 = 8$

$l=1, r=8 \rightarrow \text{area} = \min(8, 7) * 7 = 49$

...

Max Area = 49

Complexity

Time: $O(n)$

Space: $O(1)$

FAANG Tip

Two pointer reduces brute force to linear time!

Answer = 49

10 3Sum

Problem

Given an integer array $nums$, return all the triplets $[nums[i], nums[j], nums[k]]$ such that $i \neq j, i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Intuition

Sort the array. Fix one number and use two pointers to find the other two.

Optimal Approach

Sort $\rightarrow$ Fix $i \rightarrow$ Two pointers $(l, r) \rightarrow$ Skip duplicates.

Dry Run $nums = [-1, 0, 1, 2, -1, -4]$

Sorted = $[-4, -1, -1, 0, 1, 2]$

Fix $i = 0 (-4) \rightarrow$ need sum = 4 $\rightarrow$ no pair

Fix $i = 1 (-1)$

-4	-1	-1	0	1	2
0	1	2	3	4	5

$-1 + 2 = 1 \rightarrow r--$

$-1 + 1 = 0 \rightarrow$ Found $[-1, -1, 2]$

Move $l++$, $r--$ and skip duplicates

Similarly for $i = 2 \rightarrow$ Found $[-1, 0, 1]$

Answer = $[[-1, -1, 2], [-1, 0, 1]]$

Complexity

Time: $O(n^2)$

Space: $O(1)$

FAANG Tip

Handling duplicates correctly is the key to AC!

SLIDING WINDOW vs TWO POINTER

	Sliding Window	Two Pointer
Used For	Subarray/Substring problems	Pair/Triplet problems
Key Idea	Expand & Shrink window	Move pointers based on condition
Example Problems	6, 7	9, 10
Time Complexity	Usually $O(n)$	Usually $O(n)$ or $O(n \log n)$

COMMON PATTERNS

- Sliding Window
- Two Pointers
- Hashing
- Sorting
- Prefix Sum



WHEN TO USE WHAT?

- Need subarray/subsequence $\rightarrow$ Sliding Window
- Need pairs/triplets $\rightarrow$ Two Pointers
- Need fast lookup $\rightarrow$ Hashing
- Need order / easier pointers $\rightarrow$ Sorting
- Need range queries $\rightarrow$ Prefix Sum

30-SECOND REVISION

- ✓ Sliding Window = Expand & Shrink
- ✓ Two Pointer = Move Smartly
- ✓ HashSet = $O(1)$ Lookups
- ✓ Sort first when needed
- ✓ Understand the pattern, coding is easy!



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





PROBLEMS 11 - 15 (BINARY SEARCH)

11 Binary Search

Problem

Given a sorted array and a target value, return the index of target. If not found, return -1.

Intuition

Use binary search to eliminate half of the search space in each step.

Optimal Approach

Apply binary search on sorted array.

Dry Run

nums = [-1, 0, 3, 5, 9, 12], target = 9

l	r	mid	nums[mid]	Action
0	5	2	3	$3 < 9 \rightarrow l = \text{mid} + 1$
3	5	4	9	$9 == 9 \rightarrow \text{return } 4$

Answer = 4

nums = [-1, 0, 3, 5, 9, 12]

-1	0	3	5	9	12
0	1	2	3	4	5

$l = 0$ $\text{mid} = 2$ $r = 5$

Complexity

Time: $O(\log n)$

Space: $O(1)$

FAANG Tip

Always update boundaries carefully to avoid infinite loops.

12 Search in Rotated Sorted Array

Problem

Given a rotated sorted array and target, return index if found else -1.

Intuition

Use modified binary search. At least one half will always be sorted.

Optimal Approach

1. Find which half is sorted.
2. Check if target lies in that half.
3. Eliminate the other half.

Dry Run

nums = [4, 5, 6, 7, 0, 1, 2], target = 0

l	r	mid	nums[mid]	Action
0	6	3	7	Left half sorted
4	6	5	1	Target in left half
4	4	4	0	Found!

Answer = 4

Step 1

nums = [4, 5, 6, 7, 0, 1, 2]

4	5	6	7	0	1	2
0	1	2	3	4	5	6

↑

Step 2

4	5	6	7	0	1	2
0	1	2	3	4	5	6

↑

Step 3

4	5	6	7	0	1	2
0	1	2	3	4	5	6

↑

Complexity

Time: $O(\log n)$

Space: $O(1)$

FAANG Tip

Carefully handle duplicates and boundary conditions.

13 Find Peak Element

Problem

A peak element is an element that is greater than its neighbors. Return the index of any peak element.

Intuition

Use binary search. If mid is smaller than right, peak must be on right side.

Optimal Approach

Compare mid with mid+1 and decide direction.

Dry Run

nums = [1, 2, 3, 1]

l	r	mid	nums[mid]	nums[mid+1]	Action
0	3	1	2	3	go right
2	3	2	3	1	peak found

Answer = 2

Array

nums = [1, 2, 3, 1]

1	2	3	1
0	1	2	3

↑
Peak

Complexity

Time: $O(\log n)$

Space: $O(1)$

FAANG Tip

Don't worry if multiple peaks exist. Return any one of them.

14 Koko Eating Bananas

Problem

Koko has piles of bananas. She can eat k bananas per hour from one pile. Find the minimum k to finish within h hours.

Intuition

Answer lies in range $[1, \max(\text{piles})]$. Use binary search on answer.

Optimal Approach

1. Binary search on eating speed k.
2. For each k, calculate total hours.
3. If hours $\leq h \rightarrow$ try smaller k.

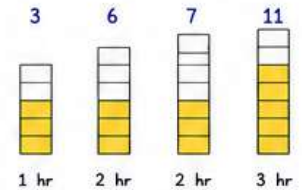
Dry Run

piles = [3, 6, 7, 11], h = 8

k(mid)	Hours	Check
6	8	$\leq 8 \rightarrow$ try smaller
3	12	$> 8 \rightarrow$ try larger
4	8	$\leq 8 \rightarrow$ try smaller

Answer = 4

Visual (Hours Calculation for k=4)



Total Hours = 8

Complexity

Time: $O(n \log m)$

(m = max pile size)

Space: $O(1)$

FAANG Tip

This is a classic binary search on answer problem!

15 Median of Two Sorted Arrays

Problem

Given two sorted arrays nums1 and nums2 of size m and n, return the median of the two sorted arrays.

Intuition

Use binary search on smaller array to partition both arrays such that left part has $(m+n+1)/2$ elements.

Optimal Approach

1. Binary search on smaller array.
2. Partition both arrays.
3. Check if partitions are valid.
4. Calculate median from partitions.

Example

nums1 = [1, 3]
nums2 = [2]

Merged = [1, 2, 3]
Median = 2.0

Answer = 2.0

Partition Diagram

nums1 $\rightarrow$ [1 | 3]
 i j

nums2 $\rightarrow$ [2 |]
 j j

Left Part Right Part
[1, 2] [3]

Left size = 2 = (3+1+1)/2 ✓

Complexity

Time: $O(\log(\min(m, n)))$

Space: $O(1)$

FAANG Tip

Always binary search on the smaller array for better complexity.

BINARY SEARCH TEMPLATE

```
int l = 0, r = n-1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (condition true) {
        // move left
        r = mid - 1;
    } else {
        // move right
        l = mid + 1;
    }
}
```

Remember: Update boundaries carefully to avoid infinite loop! ★

BINARY SEARCH USE CASES

- ✓ Sorted Array Search
- ✓ First / Last Occurrence
- ✓ Lower Bound / Upper Bound
- ✓ Binary Search on Answer
- ✓ Rotated Sorted Array
- ✓ Finding Peak Element

TIME COMPLEXITY TABLE

Case	Time
Binary Search	$O(\log n)$
Rotated Array Search	$O(\log n)$
Koko Eating Bananas	$O(n \log m)$
Median of Two Arrays	$O(\log(\min(m, n)))$

INTERVIEW CHECKLIST ✓

- ✓ Understand the problem clearly
- ✓ Think of brute force
- ✓ Identify pattern (Binary Search?)
- ✓ Define search space
- ✓ Write conditions carefully
- ✓ Dry run with examples
- ✓ Analyze time & space complexity

30-SECOND REVISION

- Binary Search works on sorted data
- Eliminate half search space
- Find the right boundaries
- Edge cases are important
- Practice variations!



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





PROBLEMS 21 - 25 (TREES)

21 Maximum Depth of Binary Tree

Problem

Find the maximum depth of a binary tree.

Intuition

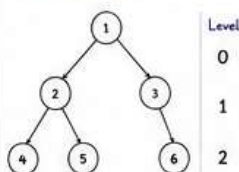
Depth = number of nodes on the longest path from root to leaf.

Optimal Approach

Use DFS (Recursion).
 $1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$

Answer = 3

Dry Run



Paths:

1 → 2 → 4 (length 2)
 1 → 2 → 5 (length 2)
 1 → 3 → 6 (length 2)
 Max Depth = 3 (levels)

Complexity

Time: $O(n)$
 Space: $O(h)$

FAANG Tip

h = height of tree. Worst case $O(n)$ for skewed tree.

22 Diameter of Binary Tree

Problem

Find the diameter (longest path between any two nodes).

Intuition

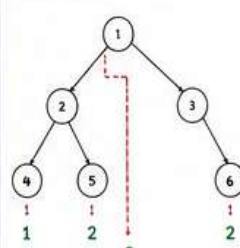
For each node, diameter can pass through it.
 Ans = $\max(\text{leftHeight} + \text{rightHeight})$

Optimal Approach

Postorder DFS.
 Return height while updating global diameter.

Answer = 4

Dry Run



4 → 2 → 1 → 3 → 6
 Diameter = 4 edges

Complexity

Time: $O(n)$
 Space: $O(h)$

FAANG Tip

Diameter may or may not pass through root.

23 Invert Binary Tree

Problem

Invert a binary tree.

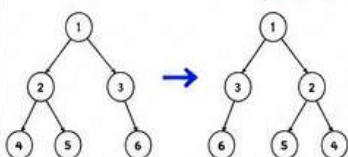
Intuition

Swap left and right subtrees for every node.

Optimal Approach

Use DFS or BFS.
 Swap children and recurse.

Dry Run



Pseudocode (DFS)

```
invert(node):
    if node == NULL: return NULL
    swap(node.left, node.right)
    invert(node.left)
    invert(node.right)
    return node
```

Inverted Tree (as shown)

Complexity

Time: $O(n)$
 Space: $O(h)$

FAANG Tip

In-place operation.
 No extra space used (except stack).

24 Lowest Common Ancestor (LCA)

Problem

Find LCA of two nodes p and q .

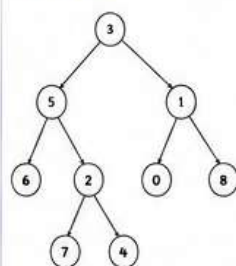
Intuition

LCA is the deepest node that has both p and q in its subtrees.

Optimal Approach

Use DFS.
 If root is NULL return NULL.
 If root is p or q return root.
 Search left and right.
 If both sides return non-NULL, current node is LCA.

Dry Run



Find LCA of 7 and 4 → Answer = 2

Answer = 2

Complexity

Time: $O(n)$
 Space: $O(h)$

FAANG Tip

If one node is ancestor of other, ancestor is LCA.

25 Validate Binary Search Tree

Problem

Check if a binary tree is a valid BST.

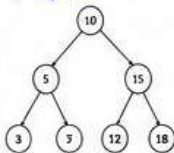
Intuition

Inorder traversal of BST gives sorted order.

Optimal Approach

Use Inorder Traversal and check if values are strictly increasing.

Dry Run



Inorder Traversal

Left → Root → Right

3, 5, 7, 10, 12, 15, 18

(sorted ✓)

Check Rules

- Left subtree < Root
- Right subtree > Root
- No duplicates (strict BST)

Answer = True (Valid BST)

Complexity

Time: $O(n)$
 Space: $O(h)$

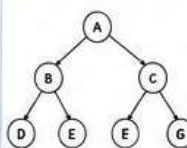
FAANG Tip

Use min/max bounds method for better optimization.

TREE TRAVERSALS QUICK RECAP

Inorder (LNR)

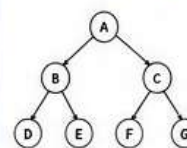
Left → Node → Right



Inorder of above tree:
 D, B, E, A, F, C, G
 $O(n)$

Preorder (NLR)

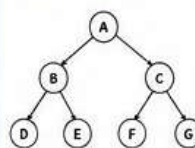
Node → Left → Right



Preorder of above tree:
 A, B, D, E, C, F, G
 $O(n)$

Postorder (LRN)

Left → Right → Node



Postorder of above tree:
 D, E, B, F, G, C, A
 $O(n)$

TREE CONCEPTS TO REMEMBER

- ✓ Root has no parent
- ✓ Leaf has no children
- ✓ Height = edges on longest path to leaf
- ✓ Full Binary Tree : Every node has 0 or 2 children
- ✓ Complete Binary Tree : All levels filled except possibly last
- ✓ Perfect Binary Tree : All internal nodes have 2 children & all leaves at same level



COMMON TREE PATTERNS

- ★ DFS / BFS
- ★ Recursion
- ★ Divide & Conquer
- ★ Backtracking
- ★ Tree DP
- ★ Path problems
- ★ Ancestor problems
- ★ Subtree problems



TIME COMPLEXITY CHEAT SHEET

Operation	Time
Traversal (Inorder/Pre/Post)	$O(n)$
Search (BST)	$O(h)$
Insert (BST)	$O(h)$
Delete (BST)	$O(h)$

n = number of nodes
 h = height of tree ($O(\log n)$ for balanced & $O(n)$ for skewed tree)

INTERVIEW CHECKLIST ✓

- ✓ Understand the problem clearly
- ✓ Draw the tree
- ✓ Think of brute force
- ✓ Look for patterns
- ✓ Choose optimal approach
- ✓ Dry run with examples
- ✓ Analyze complexity
- ✓ Handle edge cases

TREE TIPS

- 💡 Always define base case in recursion.
- 💡 Visualize the tree before coding.
- 💡 Practice more tree problems, solve daily!
- 💡 Trees are asked in almost every FAANG interview!



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

• Intuition • Approach • Dry Run • Optimal Solution • Interview Tips

Page 6



PROBLEMS 26 - 30 (GRAPHS)

26 Number of Islands

Problem

Given an $m \times n$ grid of '1's (land) and '0's (water), return the number of islands.

Intuition

Use DFS/BFS to explore all connected land cells.

Optimal Approach

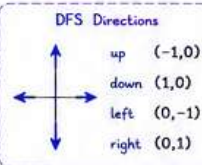
Traverse the grid. When you find a '1', perform DFS and mark all connected land as visited.

Answer = 3

Dry Run

1	1	0	0	0
1	1	0	1	0
0	0	0	1	1
0	0	0	0	0

Islands = 3



Complexity

Time: $O(mn)$
Space: $O(mn)$

FAANG Tip

Mark visited cells by changing '1' to '0' or using a visited matrix.

27 Clone Graph

Problem

Given a reference of a node in an undirected connected graph, return a deep copy (clone) of the graph.

Intuition

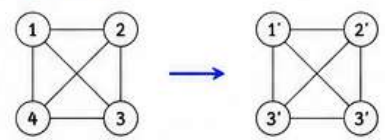
Use DFS/BFS + HashMap to map original node to cloned node.

Optimal Approach

1. Create a map to store old node $\rightarrow$ new node.
2. Traverse the graph.
3. Clone nodes and their neighbors.

Answer = Clone of Graph

Dry Run



DFS(node):

```
if node in map: return map[node]
clone = new Node(node.val)
map[node] = clone
for nei in node.neighbors:
    clone.neighbors.add(DFS(nei))
return clone
```

Complexity

Time: $O(V + E)$
Space: $O(V)$

FAANG Tip

Always use HashMap to avoid cloning same node again (infinite loop).

28 Course Schedule (Detect Cycle in Directed Graph)

Problem

There are numCourses courses labeled from 0 to $n-1$. Given prerequisites where prerequisites[i] = [a, b] means to take course a you must first take course b. Return true if you can finish all courses. Otherwise, return false.

Intuition

If graph has a cycle, you can't finish all courses.

Optimal Approach

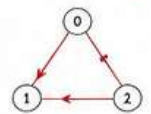
Use DFS + Cycle Detection (Recursion Stack) or Kahn's BFS (Topological Sort).

Answer = False

Dry Run (DFS)

Graph (Adjacency List)

0 $\rightarrow$ [1]
1 $\rightarrow$ [2]
2 $\rightarrow$ [0] $\leftarrow$ cycle



Cycle Found $\rightarrow$ False

Complexity

Time: $O(V + E)$
Space: $O(V)$

FAANG Tip

This is classic cycle detection in directed graph.

29 Rotting Oranges

Problem

Given an $m \times n$ grid where 0 = empty, 1 = fresh orange, 2 = rotten orange. Return the minimum time to rot all oranges. If impossible, return -1.

Intuition

Use BFS. All rotten oranges rot nearby fresh oranges in 1 minute.

Optimal Approach

Push all rotten oranges into queue and BFS.

Dry Run

Initial Grid ($t = 0$)

2	1	1
1	1	0
0	1	1

$t = 1$

2	2	1
2	1	0
0	1	1

$t = 2$

2	2	2
2	2	0
0	2	2

$t = 3$

2	2	2
2	2	0
0	2	2

Complexity

Time: $O(mn)$
Space: $O(mn)$

FAANG Tip

Multi-source BFS pattern. Add all sources (rotten oranges) to queue initially.

Answer = 3

30 Pacific Atlantic Water Flow

Problem

Given an $m \times n$ matrix heights, water can flow from cell to adjacent cell if the adjacent cell's height is less than or equal to current cell's height. Return coordinates where water can flow to both Pacific and Atlantic ocean.

Intuition

Reverse the flow. Start DFS from oceans and mark reachable cells.

Dry Run (Concept)

Pacific Ocean

1	2	2	3	5
3	2	3	4	4
2	4	5	3	1
6	7	1	4	5
5	1	1	2	4

Atlantic Ocean

Answer (example)

[[0,4],[1,3],[1,4],[2,],[3,0],[4,0]]

Steps

1. DFS from Pacific borders $\rightarrow$ mark cells.
2. DFS from Atlantic borders $\rightarrow$ mark cells.
3. Cells visited in both DFS $\rightarrow$ answer.

Complexity

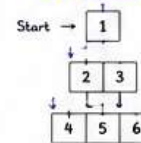
Time: $O(mn)$
Space: $O(mn)$

FAANG Tip

Reverse thinking makes hard graph problems easy!

GRAPH TRAVERSALS CHEAT SHEET

BFS (Queue)

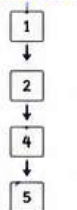


Level by Level Traversal

Use Cases

- Shortest Path in Unweighted Graph
- Level Order Traversal
- Minimum Steps Problems

DFS (Stack/Recursion)



Go Deep First

Use Cases

- Cycle Detection
- Connected Components
- Topological Sort

IMPORTANT GRAPH PATTERNS

- ✓ DFS / BFS
- ✓ Cycle Detection (Directed/Undirected)
- ✓ Topological Sort (BFS / DFS)
- ✓ Connected Components
- ✓ Multi-source BFS
- ✓ Grid DFS / BFS
- ✓ Shortest Path (BFS)
- ✓ Union Find (Disjoint Set)



COMMON MISTAKES

- ✗ Not marking visited nodes
- ✗ Forgetting to handle edge cases
- ✗ Infinite loop in cycles
- ✗ Confusing directed vs undirected
- ✗ Wrong initialization in BFS/DFS



TIME COMPLEXITY SUMMARY

Algorithm	Time Complexity
DFS	$O(V + E)$
BFS	$O(V + E)$
Topological Sort	$O(V + E)$
Dijkstra (Min Heap)	$O((V + E)\log V)$
Union Find	$O(\alpha(n))$

V = Number of Vertices

E = Number of Edges

$\alpha(n)$ = Inverse Ackermann Function

FAANG INTERVIEW CHECKLIST

- ✓ Understand the problem clearly
- ✓ Identify the graph type
- ✓ Draw the graph
- ✓ Think of brute force
- ✓ Choose optimal approach
- ✓ Dry run with example
- ✓ Analyze complexity
- ✓ Handle edge cases
- ✓ Write clean code

30-SECOND REVISION

- ★ Graph = Nodes + Edges
- ★ DFS = Go Deep
- ★ BFS = Level Order
- ★ Mark Visited
- ★ Avoid Infinite Loops
- ★ Test on Examples
- ★ Analyze Complexity



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

• Intuition • Approach • Dry Run • Optimal Solution • Interview Tips

PROBLEMS 31 - 35 (DYNAMIC PROGRAMMING)

Page 7



31 Climbing Stairs

Problem

You are climbing a staircase. You can take 1 or 2 steps at a time. In how many distinct ways can you climb to the top (n steps)?

Intuition

At each step, the ways to reach n = ways to reach (n-1) + ways to reach (n-2).

Approach

Use DP. This is Fibonacci.

Complexity Time: $O(n)$ Space: $O(n)$

Answer = 8

Dry Run (n = 5)

Step	0	1	2	3	4	5
Ways	1	1	2	3	5	8

1+1 1+2 2+3 3+5

Code (Bottom Up DP)

```
def climbStairs(n):
    if n <= 2: return n
    dp = [0]*(n+1)
    dp[0], dp[1], dp[2] = 1, 1, 2
    for i in range(3, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

32 House Robber

Problem

You are a robber planning to rob houses along a street. Adjacent houses have security system connected. Find max amount you can rob.

Intuition

If I rob this house, I cannot rob the previous one.

Approach

At each index i, choose max of:
1. Rob current + dp[i-2]
2. Skip current + dp[i-1]

Dry Run nums = [2,7,9,3,1]

Index	0	1	2	3	4
Num	2	7	9	3	1
dp[i]	2	7	11	11	12

2 max(7,2) max(9+2,7) max(3+7,11) max(1+11,11)

Code (Space Optimized)

```
def rob(nums):
    prev2, prev1 = 0, 0
    for n in nums:
        curr = max(prev1, prev2 + n)
        prev2, prev1 = prev1, curr
    return prev1
```

Complexity

Time: $O(n)$
Space: $O(1)$

Answer = 12

33 Coin Change (Minimum Coins)

Problem

Given coins of different denominations and a total amount, find the minimum number of coins that make up that amount.

Intuition

For each amount, try all coins and choose the minimum.

Approach

DP on amount.

Answer = 3

Dry Run coins = [1,2,5], amount = 11

Amount	0	1	2	3	4	5	6	7	8	9	10	11
Min Coins	0	1	1	2	2	1	2	2	3	3	2	3

1+1 1+2 1+5 2+5

Code (Bottom Up DP)

```
def coinChange(coins, amount):
    INF = float('inf')
    dp = [0] + [INF]*amount
    for a in range(1, amount+1):
        for c in coins:
            if a-c >= 0:
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

Complexity

Time: $O(\text{amount} * n)$
Space: $O(\text{amount})$

34 Longest Increasing Subsequence (LIS)

Problem

Given an array, return the length of the longest strictly increasing subsequence.

Intuition

At each index, either take it (if it increases) or skip it.

Approach

DP: dp[i] = length of LIS ending at index i.

Dry Run nums = [10,9,2,5,3,7,101,18]

Index	0	1	2	4	5	6	7
Num	10	9	2	5	7	101	18
dp[i]	1	1	1	2	2	3	4

1+2 2+5 2+7 5+7 7+101 7+18

Code ($O(n^2)$ DP)

```
def lengthOfLIS(nums):
    n = len(nums)
    dp = [1]*n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j]+1)
    return max(dp)
```

Complexity

Time: $O(n^2)$
Space: $O(n)$

Answer = 4

35 Longest Common Subsequence (LCS)

Problem

Given two strings text1 and text2, return the length of their longest common subsequence.

Intuition

If last characters match, include it and move diagonally. Else, take max of left or top.

Approach

2D DP.

Dry Run text1 = "abcde", text2 = "ace"

	a	c	e
a	0	0	0
b	0	1	1
c	0	2	2
d	0	2	2
e	0	2	3

Code (2D DP)

```
def longestCommonSubsequence(t1, t2):
    n, m = len(t1), len(t2)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if t1[i-1] == t2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][m]
```

Complexity

Time: $O(n*m)$
Space: $O(n*m)$

Answer = 3

DP STATE TRANSITION THINKING

Base Case	Smallest input (Initialize DP)	dp[0], dp[1] ...
Choices	What are the options at each state?	Take / Skip Pick / Not Pick
Transition	How current state depends on previous?	dp[i] = f(dp[i-1], dp[i-2], ...)
Answer	Where is the final answer stored?	dp[n], dp[m][n] or max(dp)

WHEN TO USE DP?

- ✓ Optimal substructure
- ✓ Overlapping subproblems
- ✓ Make choices at each step
- ✓ Can be solved recursively

COMMON DP PATTERNS

- ★ Fibonacci Pattern (Climbing Stairs)
- ★ 0/1 Choice (House Robber)
- ★ Unbounded Choices (Coin Change)
- ★ Sequence Pattern (LIS, LCS)
- ★ 2D Grid DP (LCS, Edit Distance)



TIME COMPLEXITY REFERENCE

Pattern	Time
1D DP	$O(n)$
2D DP	$O(n*m)$
DP + Nested Loop	$O(n^2)$
DP + 3 Loops	$O(n^3)$

INTERVIEW CHECKLIST ✓

- ✓ Understand the problem clearly
- ✓ Identify DP or not
- ✓ Define state and choices
- ✓ Write the relation / transition
- ✓ Handle base cases
- ✓ Code + Dry run + Test cases

COMMON MISTAKES ✗

- ✗ Wrong base cases
- ✗ Incorrect state definition
- ✗ Overlapping subproblems not identified
- ✗ Forgetting to return answer
- ✗ Index out of bound errors



30-SECOND REVISION

- DP = State + Transition
- Think Recursively → Memorize
- Draw the DP table
- Solve small cases first
- Practice & Observe Patterns



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

• Intuition • Approach • Dry Run • Optimal Solution • Interview Tips

PROBLEMS 36 - 40 (BACKTRACKING & RECURSION)

Page 8



36 Subsets

Problem

Given an array nums of unique integers, return all possible subsets (power set).

Intuition

At each index, you have two choices: pick it or not pick it.

Approach

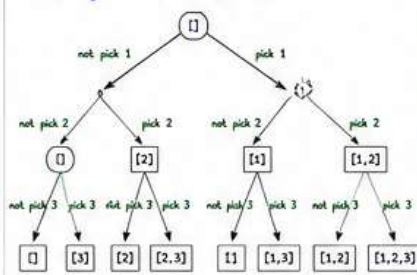
Backtracking (DFS).
Explore both choices.

Code (Backtracking)

```
def res = []
def backtrack(i, path):
    res.append(path[:])
    for j in range(i, len(nums)):
        path.append(nums[j])
        backtrack(j+1, path)
        path.pop()
    backtrack(i, path)
    return res
```

Answer = [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]

Dry Run



Complexity

Time: $O(2^n \cdot n)$
Space: $O(n)$

FAANG Tip

Classic backtracking problem. Focus on recursion tree!

37 Permutations

Problem

Given an array nums of distinct integers, return all the possible permutations.

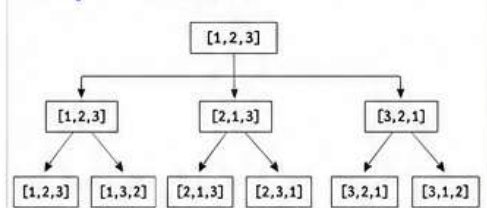
Intuition

Fix one number, then permute the rest.

Approach

Backtracking + Swapping.
Swap current index with every other index.

Dry Run



Code (Backtracking + Swap)

```
def permute(nums):
    res = []
    def backtrack(i):
        if i == len(nums):
            res.append(nums[:])
            return
        for j in range(i, len(nums)):
            nums[i], nums[j] = nums[j], nums[i]
            backtrack(i+1)
            nums[i], nums[j] = nums[j], nums[i]
        backtrack(i)
    backtrack(0)
    return res
```

Complexity

Time: $O(n \cdot n!)$
Space: $O(n)$

FAANG Tip

Don't forget to backtrack (swap back)!

Answer = [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1], [3, 1, 2]]

38 Combination Sum

Problem

Given candidates and target, find all unique combinations where candidate numbers sum to target. You may choose the same number multiple times.

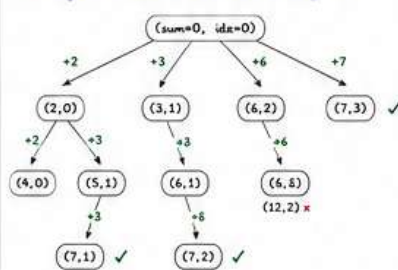
Intuition

Move forward in array and keep adding until sum reaches target.

Approach

Backtracking with start index (no revisits to previous indices).

Dry Run



Code (Backtracking)

```
def combinationSum(candidates, target):
    res = []
    def backtrack(i, s, path):
        if s == target:
            res.append(path[:])
            return
        if s > target or i == len(candidates):
            return
        backtrack(i, s + candidates[i], path + [candidates[i]])
        backtrack(i+1, s, path)
    backtrack(0, 0, [])
    return res
```

Complexity

Time: Exponential
Space: $O(\text{target} / \min(\text{candidates}))$

FAANG Tip

Two choices at each step: take it again or move forward.

Answer = [[2, 2, 3], [7]]

39 Word Search

Problem

Given a board of letters and a word, return true if word exists in the grid. The word can be formed by sequentially adjacent cells (up, down, left, right).

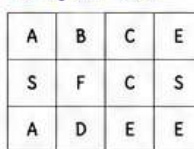
Intuition

Try each cell as start and perform DFS.

Approach

Backtracking + Visited (DFS in 4 directions).

Dry Run



Code (DFS Backtracking)

```
def exist(board, word):
    m, n = len(board), len(board[0])
    def dfs(i, j, k):
        if k == len(word):
            return True
        if i < 0 or i > m or j < 0 or j > n or board[i][j] != word[k]:
            return False
        tmp, board[i][j] = board[i][j], '#'
        found = (dfs(i-1, j, k+1) or dfs(i+1, j, k+1) or
                dfs(i, j-1, k+1) or dfs(i, j+1, k+1))
        board[i][j] = tmp
        return found
    for i in range(m):
        for j in range(n):
            if dfs(i, j, 0):
                return True
    return False
```

Complexity

Time: $O(m \cdot n \cdot 4^L)$
Space: $O(L)$
(L = length of word)

FAANG Tip

Mark visited as '#' to avoid revisiting.

Answer = True

40 N-Queens

Problem

Place N queens on an NxN chessboard so that no two queens threaten each other. Return all distinct solutions.

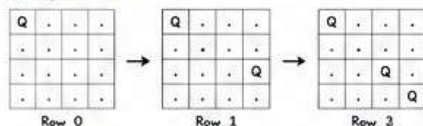
Intuition

Place one queen per row. Check column and diagonals.

Approach

Backtracking + Validity Check.

Dry Run



Code (Backtracking)

```
def solveNQueens(n):
    res, cols, d1, d2, board = [], set(), set(), set(), []
    def backtrack(r):
        if r == n:
            res.append(["".join(row for row in board)])
            return
        for c in range(n):
            if c in cols or r+c in d1 or r-c in d2:
                continue
            cols.add(c); d1.add(r+c); d2.add(r-c); board.append("." * r + "Q" + "." * (n-r-1))
            backtrack(r+1)
            board.pop(); cols.remove(c); d1.remove(r+c); d2.remove(r-c)
    backtrack(0)
    return res
```

Answer = 2 Solutions (for N=4)

["Q...","...Q",".Q..","..Q."]
["Q...","Q..","..Q.",".Q.."]

Complexity

Time: $O(N!)$
Space: $O(N)$

FAANG Tip

Use sets to track columns and diagonals.

BACKTRACKING QUICK GUIDE

Backtracking Template

```
def backtrack(choices):
    if isSolution():
        saveSolution()
        return
    for choice in choices:
        if isValid(choice):
            makeChoice(choice)
            backtrack(choices)
            undoChoice(choice)
```

When to Use?

- Generate all possibilities (permute / subset)
- Find all valid solutions
- Constraint satisfaction problems
- Optimization problems (combination / path)

Backtracking Tips

- Make a choice → Explore → Undo choice
- Always define base case clearly
- Prune early for optimization
- Visualize the recursion tree!

COMMON BACKTRACKING PATTERNS

- Subset / Subsequence
- Permutations
- Combination Sum
- N-Queens / Sudoku
- Word Search / Grid Problems
- Maze / Path Problems



TIME COMPLEXITY CHEAT SHEET

Problem Type	Time Complexity
Subset (2^n)	$O(2^n \cdot n)$
Permutation ($n!$)	$O(n!)$
Combination Sum	Exponential
N-Queens	$O(N!)$
Word Search ($m \cdot n$)	$O(m \cdot n \cdot 4^L)$

INTERVIEW CHECKLIST

- Understand the problem clearly
- Identify the choices
- Define base case
- Check constraints early
- Backtrack (undo) properly
- Test with small examples
- Analyze time & space complexity

COMMON MISTAKES

- Not backtracking (undo step)
- Incorrect base case
- Missing constraints
- Revisiting same state
- Overlooking optimization

30-SECOND REVISION

- Backtrack = Choose → Explore → Undo
- Use recursion for nature of problem
- Prune & optimize
- Visualize the recursion tree
- Practice more problems!



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





★ Practice
Consistently
Crack
FAANG!



DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

• Intuition • Approach • Dry Run • Optimal Solution • Interview Tips

PROBLEMS 41 - 45 (HEAP, STACK & QUEUE)

41 Top K Frequent Elements

Problem

Given an array `nums` and an integer `k`, return the top `k` frequent elements.

Intuition

Use frequency map + Heap.

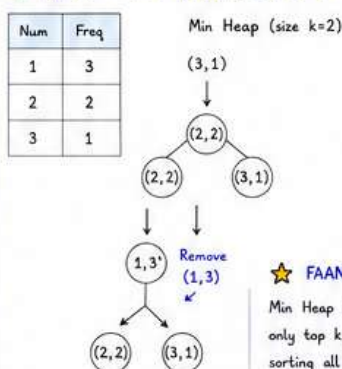
Approach

Count frequency using HashMap. Push (freq, num) in Min Heap of size `k`.

Complexity

Time: $O(n \log k)$
Space: $O(n)$

Dry Run `nums = [1,1,1,2,2,3], k = 2`



★ FAANG Tip

Min Heap keeps only top `k`, avoid sorting all unique elements.

Answer = [1,2] (any order)

42 Kth Largest Element in an Array

Problem

Find the `k`th largest element in an unsorted array.

Intuition

Use Min Heap of size `k` or Quickselect.

Approach

Push first `k` elements in heap. For remaining elements, replace if greater than top.

Dry Run `nums = [3,2,1,5,6,4], k = 2`

Min Heap (size `k=2`)

1) Add 3,2



2) Next = 1 (<2) ignore

3) Next = 5 (>2) replace



4) Next = 6 (>3) replace



5) Next = 4 (<5) ignore

Complexity

Time: $O(n \log k)$
Space: $O(k)$

★ FAANG Tip

Quickselect gives avg $O(n)$ time. Good for large inputs.

Answer = 5

43 Min Stack

Problem

Design a stack that supports push, pop, top and retrieving the minimum element in $O(1)$ time.

Intuition

Use an extra stack to store minimum elements.

Approach

Push to `minStack` only if `current <= min` so far.

Dry Run

Operation	Stack	MinStack	getMin()
push(3)	[3]	[3]	3
push(5)	[3,5]	[3]	3
push(2)	[3,5,2]	[3,2]	2
push(1)	[3,5,2,1]	[3,2,1]	1
pop()	[3,5,2]	[3,2]	2
top()	[3,5,2]	[3,2]	2
getMin()	-	-	2

★ FAANG Tip

Interviewer may ask: Can you do it using only one stack? → Encode values.

Answer = All operations in $O(1)$

44 Daily Temperatures

Problem

Given an array `temperatures`, return an array `answer` such that `answer[i]` is the number of days you have to wait until a warmer temperature.

Intuition

Use Monotonic Decreasing Stack. Keep indices.

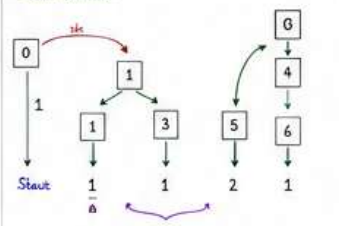
Approach

Pop while current temp is greater. Calculate distance.

Dry Run `temps = [73,74,75,71,69,72,76,73]`

Index	0	1	2	3	4	5	7
Temp	73	74	75	71	69	72	73

Stack (indices)



Complexity

Time: $O(n)$
Space: $O(n)$

★ FAANG Tip

Monotonic stack is key. Don't forget to store indices!

Answer = [1,1,1,2,1,1,0,0]

45 Valid Parentheses

Problem

Given a string `s` containing '()', '{}', '[]', determine if the input string is valid.

Intuition

Use Stack. Push opening brackets, pop on closing and match.

Approach

If mismatch or extra closing, return false. Stack must be empty at end.

Complexity

Time: $O(n)$
Space: $O(n)$

Dry Run `s = "{[()]}"`

Char	Action	Stack	Valid?
{	push	{	-
[	push	{ [	-
(	push	{ [(	-
)	pop (	{ [	-
]	pop [	{	-
}	pop {	-	True

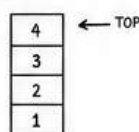
★ FAANG Tip

Interviewer may add invalid cases like "()", "{()}", "{[{()]}" to test edge cases.

Answer = True

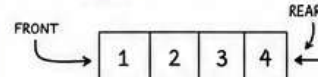
STACK & QUEUE QUICK NOTES

Stack



- LIFO (Last In First Out)
- Used in recursion, backtracking, parsing, monotonic stack
- Operations: push, pop, peek

Queue



- FIFO (First In First Out)
- Used in BFS, scheduling, queues, sliding window
- Operations: enqueue, dequeue, front/peek

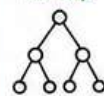
Monotonic Stack

Increasing Stack: Keep smaller at bottom (used in Next Smaller)

Decreasing Stack: Keep larger at bottom (used in Next Greater)

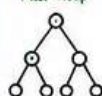
HEAP CHEAT SHEET

Min Heap



Parent ≤ Children
Peek = Smallest

Max Heap



Parent ≥ Children
Peek = Largest

HEAP USE CASES

- ✓ Top K Frequent Elements
- ✓ Kth Largest/Smallest
- ✓ Merge K Sorted Lists
- ✓ Meeting Rooms II
- ✓ Task Scheduling

STACK USE CASES

- ✓ Valid Parentheses
- ✓ Min Stack
- ✓ Daily Temperatures
- ✓ Next Greater Element
- ✓ Expression Evaluation
- ✓ Backtracking
- ✓ DFS (Iterative)

QUEUE USE CASES

- ✓ BFS
- ✓ Sliding Window (Max/Min)
- ✓ Level Order Traversal
- ✓ Task Scheduling
- ✓ Producer Consumer Problem

COMPLEXITY REMINDER

Operation	Time Complexity
Stack push / pop / peek	$O(1)$
Queue enqueue / dequeue	$O(1)$
Heap push / pop / peek	$O(\log n)$
Build Heap (n items)	$O(n)$

INTERVIEW CHECKLIST ✓

- ✓ Clarify the problem
- ✓ Think of brute force
- ✓ Optimize (Space + Time)
- ✓ Handle edge cases
- ✓ Dry run with example
- ✓ Write clean code
- ✓ Check complexity
- ✓ Test more cases

COMMON MISTAKES ✗

- ✗ Forgetting to handle empty stack/queue
- ✗ Wrong order of push/pop
- ✗ Not checking all brackets
- ✗ Off-by-one in indices
- ✗ Not considering duplicates

30-SECOND REVISION ⚡

- Stack = LIFO
- Queue = FIFO
- Heap gives sorted access in $\log n$
- Monotonic Stack = Powerful Pattern
- Dry Run = Must before coding

Consistency Today

Confidence Tomorrow

Crack FAANG 🚀



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!





DSA Interview Handbook

Top 50 Most Asked FAANG DSA Problems

• Intuition • Approach • Dry Run • Optimal Solution • Interview Tips

PROBLEMS 46 - 50 (ADVANCED TOPICS)

46 LRU Cache

Problem

Design an LRU (Least Recently Used) Cache with get and put in O(1) time.

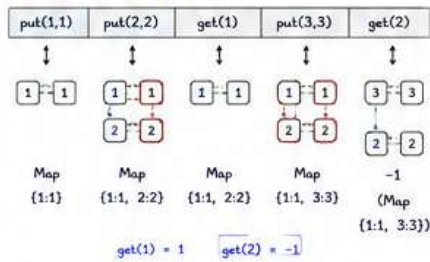
Intuition

Use Doubly Linked List + HashMap.
DLL maintains order of usage.

Approach

Most recent at head, least recent at tail.

Dry Run capacity = 2



```
Code (Python)
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.dll = DoublyLinkedList() # head <--> tail

    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self.dll.moveToHead(node)
        return node.val

    def put(self, key, value):
        if key in self.map:
            node = self.map[key]
            node.val = value
            self.dll.moveToHead(node)
        else:
            if len(self.map) == self.cap:
                tail = self.dll.removeTail()
                del self.map[tail.key]
            node = Node(key, value)
            self.dll.addToHead(node)
            self.map[key] = node
```

Complexity

Time: O(1)
Space: O(n)

FAANG Tip

This is a must know for cache, system design & real world implementations.

Answer = Implement LRU with DLL + HashMap

48 Trapping Rain Water

Problem

Given n non-negative integers representing an elevation map, compute how much water it can trap after raining.

Intuition

Water trapped at i = min(left max, right max) - height[i]

Approach

Use Two Pointers. Move from both ends maintaining max heights.

Dry Run height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

	0	1	2	3	4	5	6	7	8	9	10	11
Height	0	1	0	2	0	1	1	3	2	1	2	1
Left Max	0	1	1	2	2	2	2	3	3	3	3	3
Right Max	3	3	3	3	3	3	3	3	2	2	2	1
Water	0	0	1	0	1	2	1	0	0	1	0	0

Total Water = 6

```
Code (Python)
def trap(height):
    l, r = 0, len(height)-1
    leftMax, rightMax = 0, 0
    water = 0
    while l < r:
        if height[l] < height[r]:
            if height[l] >= leftMax:
                leftMax = height[l]
            else:
                water += leftMax - height[l]
            l += 1
        else:
            if height[r] >= rightMax:
                rightMax = height[r]
            else:
                water += rightMax - height[r]
            r -= 1
    return water
```

Complexity

Time: O(n)
Space: O(1)

FAANG Tip

Two Pointer approach is optimal.

Answer = 6 units

50 Median of Two Sorted Arrays

Problem

Given two sorted arrays nums1 and nums2 of size m and n, return the median of the two sorted arrays.

Intuition

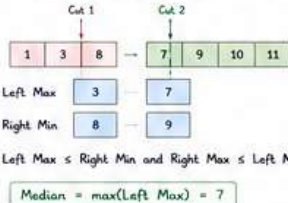
Reduce to finding k-th element in two sorted arrays.

Approach

Use Binary Search on shorter array. Partition arrays.

Dry Run nums1 = [1,3,8], nums2 = [7,9,10,11]

Total = 7 (odd) → find 4th element



Complexity

Time: O(log(min(m,n)))
Space: O(1)

FAANG Tip

This is the optimal logarithmic solution. Binary search makes it fast!

Answer = 7

```
Code (Python)
# Binary search on shorter array
def findMedianSortedArrays(nums1, nums2):
    m, n = len(nums1), len(nums2)
    low, high = 0, m
    while low < high:
        k1 = (low + high) // 2
```

47 Merge k Sorted Lists

Problem

Merge k sorted linked lists and return it as one sorted list.

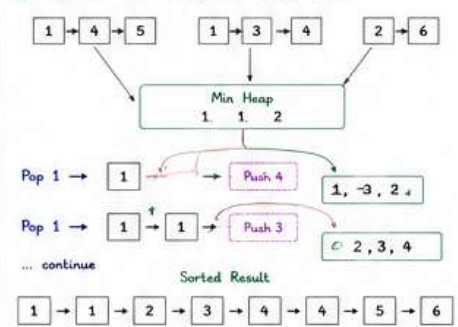
Intuition

Always pick the smallest node among all lists.

Approach

Use Min Heap (Priority Queue) to get the smallest node in O(log k).

Dry Run lists = [[1,4,5],[1,3,4],[2,6]]



```
Code (Python)
import heapq
def mergeKLists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode()
    curr = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity

Time: O(N log k)
Space: O(k)

FAANG Tip

k can be large. Heap avoids comparing all nodes every time.

Answer = Use Min Heap to merge in O(N log k)

49 Serialize and Deserialize Binary Tree

Problem

Serialize a binary tree to a string and deserialize it back to the same tree.

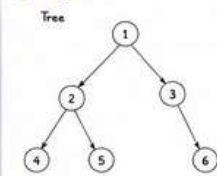
Intuition

Use Preorder traversal. Add null markers for empty children.

Approach

Preorder + '#' for null. Split by space while deserializing.

Dry Run



Serialized (Preorder)

1 2 4 # 5 # 3 6 # #

Deserialization

Read 1 → root
Read 2 → left of 1
Read 4 → left of 2
→ null
→ null
Read 5 → right of 2
...

```
Code (Python)
# Serialize
def serialize(root):
    res = []
    def dfs(node):
        if not node:
            res.append('#')
            return
        res.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ' '.join(res)

# Deserialize
def deserialize(data):
    vals = data.split()
    def build():
        val = vals.pop(0)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()
```

Complexity

Time: O(n)
Space: O(n)

FAANG Tip

Make sure your serialization is unique!

Answer = Unique serialization using preorder + nulls

QUICK REFERENCE CHEAT SHEET

DATA STRUCTURES

- Array
- String
- HashMap / Set
- Heap (Min / Max)
- Tree (Binary Tree, BST)
- Graph (Adj List, Matrix)



COMMON COMPLEXITIES

- O(1) Constant
- O(log n) Logarithmic
- O(n) Linear
- O(n log n) Linearithmic
- O(n^2) Quadratic
- O(2^n) Exponential
- O(n!) Factorial



REMEMBER

★ Practice Consistently → Build Confidence → Crack FAANG

INTERVIEW TIPS

- ✓ Clarify the problem
- ✓ Think out loud
- ✓ Start with brute force
- ✓ Optimize step by step
- ✓ Handle edge cases
- ✓ Dry run with examples
- ✓ Analyze complexity
- ✓ Write clean code



SAVE THIS POST
for future revision!



COMMENT "DSA"
for full PDF in your DM!



SHARE IT
with your friends!

Keep Practicing,
Keep Improving!

